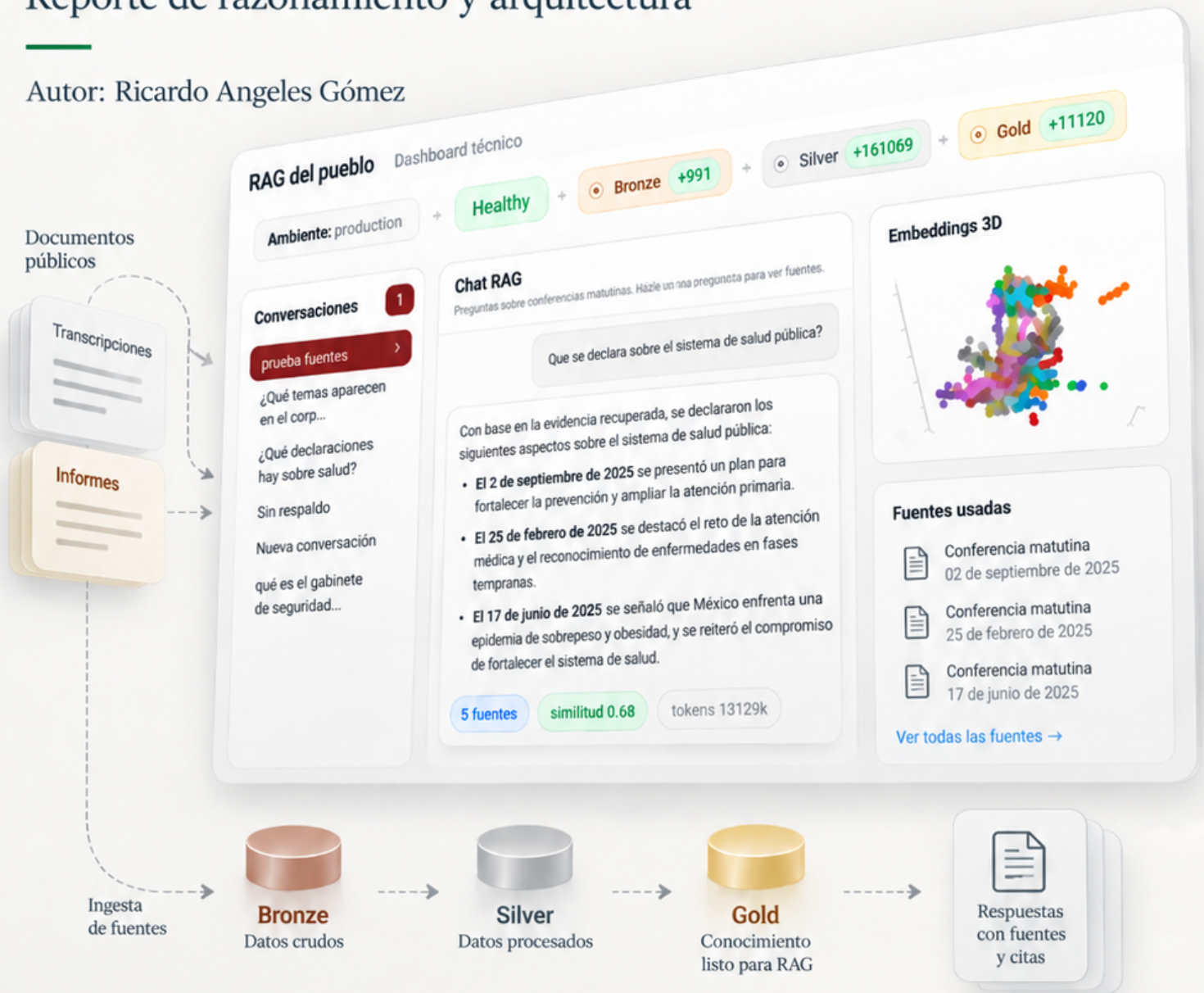


RAG del pueblo

Agente de indagación de las conferencias matutinas del Gobierno Federal de México

Trabajo final Diplomado de Ingeniería en IA.
Reporte de razonamiento y arquitectura

Autor: Ricardo Angeles Gómez



Recuperación de información



Memoria conversacional



Arquitectura escalable



Al servicio de la ciudadanía

RAG del pueblo

Agente de investigación de las conferencias matutinas del Gobierno de México.

Pipeline de datos con arquitectura medallón (Bronze → Silver → Gold), memoria conversacional aislada y tres herramientas de solo lectura, con evidencia rastreable.

URL productiva: `https://rag-del-pueblo-iens6os2ba-uc.a.run.app`

1. Problema y usuarios

Las conferencias matutinas contienen declaraciones dispersas entre fechas, participantes y temas. Una búsqueda convencional encuentra fragmentos parecidos, pero no ayuda a:

- continuar una investigación entre turnos;
- distinguir una sesión de otra;
- explorar temas emergentes del corpus;
- explicar qué herramienta se usó;
- **negarse a concluir** cuando el corpus no respalda una afirmación.

Usuarios: periodistas y analistas que necesitan localizar evidencia verificable y entender su procedencia. En esta entrega: dos usuarios demo.

Valor: reducir el tiempo para localizar declaraciones y convertir resultados dispersos en una respuesta rastreable. El agente no sustituye el criterio periodístico: organiza evidencia, cita procedencia y se niega a concluir sin respaldo.

2. Decisiones de arquitectura

- **Sin frameworks agénticos.** FastAPI + Pydantic + Vue + PostgreSQL/pgvector + Ollama/EmbeddingGemma + llama.cpp (local) y Gemini (producción).
- **Agente = capa pequeña** de planificación (plan JSON validado por Pydantic) + ejecución segura sobre los servicios existentes.

- **Tres tools de solo lectura** (allowlist): `buscar_declaraciones`, `explorar_temas`, `consultar_cluster`.
- **Autenticación** con PyJWT (HS256, cookie HttpOnly + SameSite=Strict) + Argon2 + CSRF.
- **Memoria aislada** por conversación y por usuario (propiedad derivada del JWT; una conversación ajena responde `404`), retención de 30 días.
- **Despliegue** Cloud Run + Neon (TLS/pooling) + Gemini mediante Terraform; corpus productivo reindexado con `gemini-embedding-001` (768d).

3. Resultados reales

Chequeo	Resultado
Backend <code>pytest</code>	679 passed · cobertura 96.33% ($\geq 90\%$)
Frontend	<code>pnpm typecheck</code> + 47 unit tests
Calidad	<code>ruff</code> y <code>ty</code> sin errores; <code>make security</code> 16/16
Evaluación RAG (LLM-as-a-Judge)	50 preguntas · fidelidad 93.51% · relevancia 99.53% · cobertura 73.09%
Corpus productivo	11,120 chunks reindexados con Gemini (768d) · 52 clusters etiquetados · 11,120 puntos 3D
Costo mensual estimado	~USD 0.00 (<code>gcp_cost.py</code> determinista, dentro del nivel gratuito)

4. Evidencia en la URL productiva

4.1 Acceso restringido (login)

Todo el dashboard y la API exigen autenticación; solo el login y `GET /health` son públicos.

RAG del pueblo

Acceso restringido. Usa las credenciales demo entregadas en el PDF de la entrega.

Usuario

Contraseña

Entrar

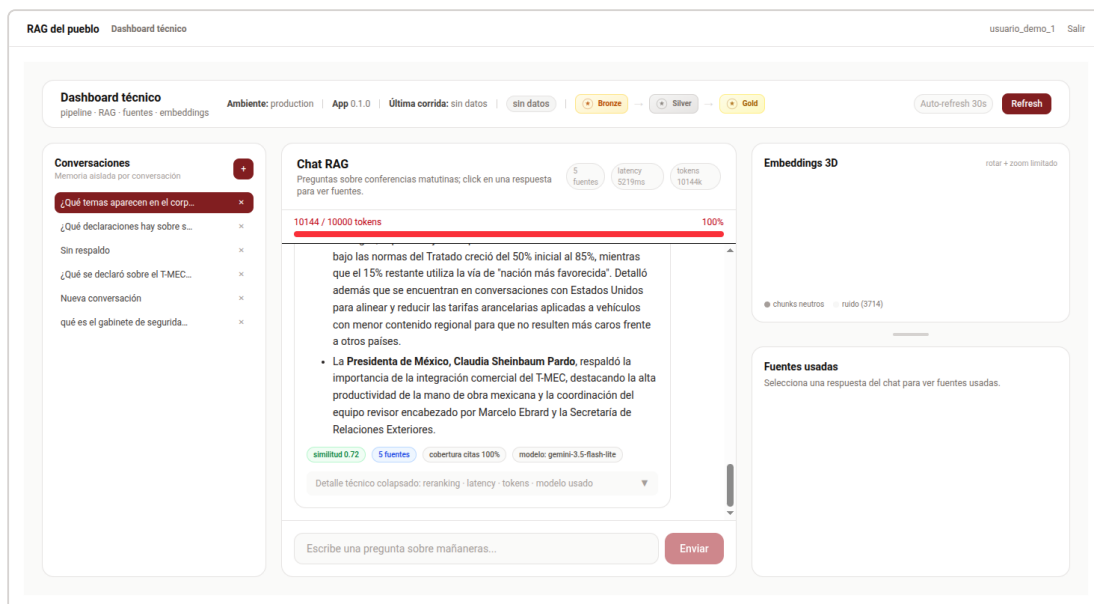
En producción el nivel gratuito de Gemini puede usar los datos enviados para mejorar productos de Google: no introduzcas investigaciones confidenciales.

Pantalla de login

Figura 1 — Login. El dashboard redirige a `/login` sin sesión.

4.2 Turno del agente con evidencia y traza

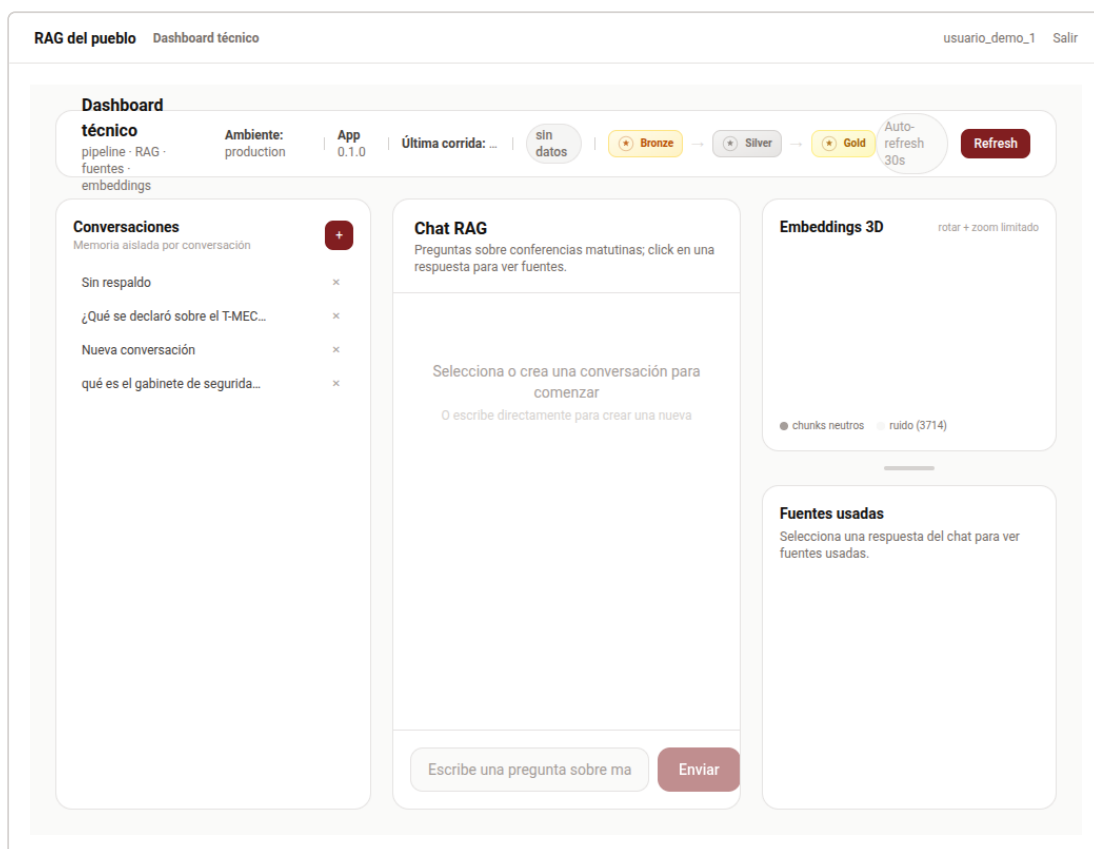
Pregunta: «¿Qué se declaró sobre el T-MEC?» → el agente invocó `buscar_declaraciones` y respondió con 5 fuentes citadas (fecha, participante, cita), modelo `gemini-3.5-flash-lite`, latencia y tokens visibles.



Turno del agente con fuentes

Figura 2 – Turno del agente con 5 fuentes citadas y métricas.

4.3 Dashboard con clusters y embeddings 3D

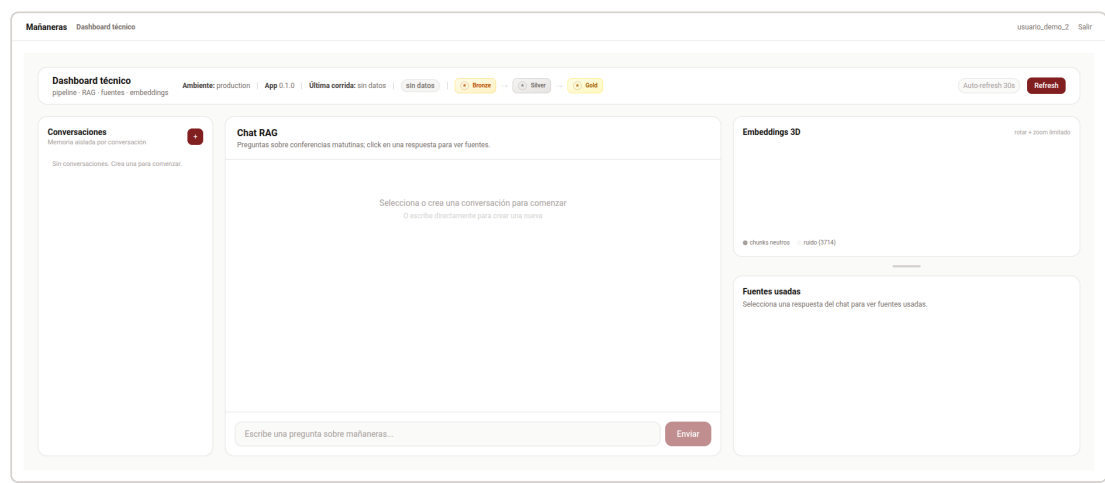


Dashboard usuario 1

Figura 3 – Dashboard del usuario demo 1: memoria conversacional, scatter 3D y clusters.

4.4 Aislamiento entre usuarios

El usuario demo 2 no ve las conversaciones del usuario 1 (lista vacía) y las llamadas cruzadas por identificador devuelven 404 .



Dashboard usuario 2 aislado

Figura 4 – Usuario demo 2: sin conversaciones ajenas (aislamiento verificado).

5. Negativa sin evidencia

Pregunta: «¿Qué dijo la presidenta sobre un viaje a Marte en 1999?». El agente se negó a concluir:

No encontré evidencia suficiente en el corpus para sostener esa conclusión.

Traza: buscar_declaraciones → 0 resultados (status ok , 507 ms), refusal=true .

6. Credenciales demo (únicamente en este documento)

Usuario	Contraseña
usuario_demo_1	YeB2aV2)nQ##28i8VjZ2hBro
usuario_demo_2	fiX9}j?cFZr:Ae?r6da>3JDP

7. Comandos de reproducción

```
# Despliegue (Terraform, estado remoto en GCS)
cd infra/terraform && terraform init && terraform plan && terraform apply

# Reindexación productiva (corpus Gemini hacia Neon)
NEON_DATABASE_URL=<secret> GEMINI_API_KEY=<secret> \
  uv run python -m lakehouse pipeline reindex-production

# Visuales (clusters + 3D sobre Gemini)
NEON_DATABASE_URL=<secret> uv run python -m lakehouse pipeline sync-production-vi

# Observabilidad (corridas del pipeline local -> Neon)
NEON_DATABASE_URL=<secret> uv run python -m lakehouse pipeline sync-production-ob

# Evaluación RAG real (LLM-as-a-Judge)
uv run python -m lakehouse evaluate-rag

# Costo determinista (~USD 0)
python3 scripts/gcp_cost.py --config scripts/gcp_cost.example.json
```

8. Observaciones personales

(Escribe aquí tus observaciones sobre el funcionamiento, la interfaz, la calidad de las respuestas, la evidencia, o cualquier hallazgo durante la demo.)

9. Experiencia y aprendizajes

(Describe aquí tu experiencia desarrollando y desplegando el proyecto: qué fue lo más difícil, qué aprendiste, qué harías distinto.)

10. Notas técnicas y correcciones durante el desarrollo

Durante el smoke productivo se detectaron y corrigieron varios problemas:

- **Gráfico 3D no renderizaba en producción.** `echarts@6` + `echarts-gl@2.1` lanzaban `Invalid expression` al crear `scatter3D`, agravado por la CSP que bloqueaba `eval` (claygl usa `new Function`). Se bajó a `echarts@5.6` y se permitió `'unsafe-eval'` en la CSP solo para ese gráfico.
- **Corridas del pipeline no visibles en producción.** Se añadió `pipeline sync-production-observability` para replicar `observability.pipeline_runs` local → Neon.
- **Fuentes no persistidas en memoria.** Se guardan `sources` (con `cluster_id` y `embedding_3d`) y `latency_ms` en cada mensaje, de modo que al reabrir una conversación se recuperan las fuentes y se resalta el cluster en el 3D.

Proyecto académico — diplomado. Ningún secreto (JWT/CSRF, keys o contraseñas) se almacena en el repositorio; vive en Secret Manager. Las credenciales demo se entregan únicamente en este documento.